



AI STUDENT COLLECTIVE MADRID

Buscador de Convalidaciones

Fase 1 · MVP — plan de trabajo

Francisco Jimeno · 100499080@alumnos.uc3m.es

Madrid, 1 de octubre de 2026

1 grado UC3M + 1 universidad de destino, en ~2 meses

Objetivo: validar el flujo completo de principio a fin (datos → IA → web → PDF) antes de escalar. Presupuesto: 0 €. Equipo de 6 a tiempo parcial.

Qué entra

- Esquema común de asignatura: nombre, créditos → ECTS, contenidos, idioma, semestre.
- Scraper UC3M del grado piloto y un conector para la universidad piloto.
- Carga del histórico oficial MNE (XLS) y enlace con el catálogo del destino.
- Matching semántico con umbral calibrado con datos reales.
- App Streamlit pública con exportación de la propuesta en PDF.

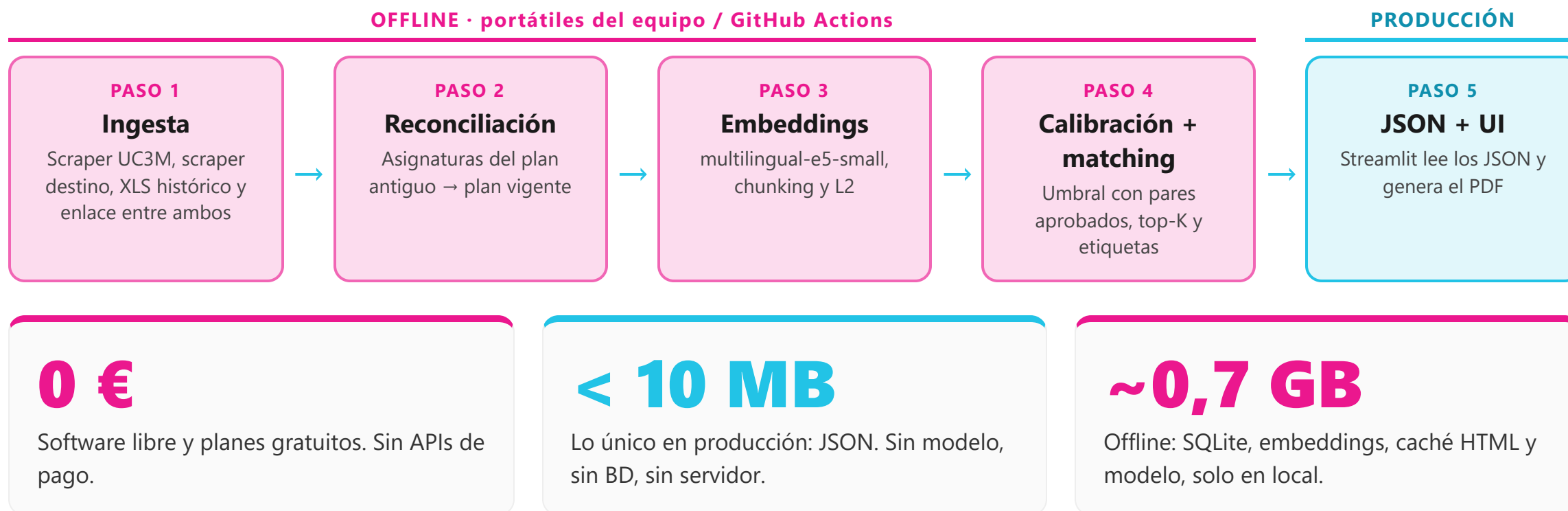
Qué no entra

- Otros grados y otras universidades.
- Propuestas combinadas (2 asignaturas destino → 1 UC3M): solo pares 1:1.
- Búsqueda libre ("pega tu temario"): no hay modelo en producción.
- Refresco automático programado: basta un script (`make refresh`).

El piloto (grado + universidad) se elige en la Semana 1 con datos: el que tenga más pares históricos **con catálogo web que publique descripciones**.

Pipeline en 5 pasos: precalcular y servir **estático**

Las consultas posibles son finitas (grado × universidad), así que todo se calcula antes. Cada paso es un script independiente y testeable.



Tres fuentes y un **enlace** entre ellas

El XLS histórico solo trae nombres y códigos. Para comparar contenidos hay que enlazar cada fila con el catálogo scrapeado.

UC3M requests + BS4

- Enumeración con endpoints JSON: `findPlanes`, `findAsignaturas`.
- Ficha: `generaFicha?...&anio=&idioma=1|2`, HTML estático.
- Secciones por diccionario de títulos ES/EN (algunas son opcionales).
- Piloto: ~270 peticiones, ~7 min a 1,5 s/petición.
- Cambios: fecha de actualización + SHA-256 del texto.

Destino 1 YAML por universidad

- `requests` + BS4, o `Playwright` si el catálogo es una SPA.
- Selectores, rate limit y factor `credits_to_ects` en el YAML.
- Salida normalizada al esquema común.
- Caché HTML local (nunca se sube al repo).

Histórico + enlace pandas + xlrd

- 3 XLS oficiales MNE, ~14.700 filas, validadas con Pydantic.
- Normalización y deduplicación entre cursos; relaciones n:m con `group_id`.
- Enlace: 1) código normalizado, 2) nombre con `rapidfuzz` ≥ 90 , 3) si no, `unlinked`.
- La tasa de enlace decide también el piloto.

Del plan **antiguo** al plan vigente

Muchas filas del histórico usan códigos de planes que ya no existen. Sin este paso no sirven ni para calibrar ni para etiquetar precedentes. Se hace en cascada, de lo más barato a lo más caro.

1

Mismo código

El código existe en el plan vigente (`findAsignaturas` con el año actual).

Match directo

2

Mismo nombre normalizado

Sin tildes, mayúsculas ni espacios extra.

Match directo

3

Similitud del temario

Ficha antigua (`año` del XLS) frente a fichas del plan vigente. $\tau_{\text{alto}} = 0,85 \cdot \tau_{\text{bajo}} = 0,65$ (se ajustan con la revisión manual de la Semana 4).

$\geq \tau_{\text{alto}}$ auto · entre ambos revisión
· $< \tau_{\text{bajo}}$ sin equivalente

4

Desdoblamientos y fusiones (1→2, 2→1)

Casos ambiguos.

Siempre revisión manual

Salida: tabla `old_code → new_code` con el método usado (code / exact / embedding / manual).

Cada asignatura como un **vector** de 384 dimensiones

Dos asignaturas con temarios parecidos dan vectores cercanos, aunque estén en idiomas distintos. Se compara por similitud coseno.

Modelo	<code>intfloat/multilingual-e5-small</code> con <code>sentence-transformers</code> , en CPU	Multilingüe, 512 tokens de ventana. MiniLM solo lee 128 (~1/5 de una ficha).
Texto de entrada	<code>"query: {nombre}. {Objetivos}. {Programa}"</code> , versión EN si existe	Competencias, evaluación y metodología fuera: solo añaden ruido.
Textos largos	Trozos de ~400 tokens → media de sus embeddings	Una ficha real ronda 600-700 tokens: sin trocear se perdería el final.
Campos comparables	Mismo tipo de campo en ambos lados	Temario largo frente a descripción de 3 líneas baja el coseno.
Normalización	L2 (<code>normalize_embeddings=True</code>)	El producto escalar pasa a ser coseno real.
Búsqueda	Producto matricial con <code>numpy</code>	Con cientos de asignaturas no hace falta un índice vectorial.
Benchmark (Semanas 5-6, opcional)	e5-small frente a MiniLM-L12 y e5-base	Se elige el de mejor Recall@5, no a priori.

Un umbral que sale de los **datos**, no fijado a mano

Se usan solo los pares del histórico reconciliados y enlazados: los únicos en los que se conoce el contenido de los dos lados.

Calibración y evaluación

Split 80/20 por asignatura UC3M: la misma asignatura nunca está en train y test.

Coseno de cada par aprobado en train.

Filtro de outliers: se descartan los pares por debajo de mediana – 3·MAD (y se revisan).

Umbral = percentil 5-10 de lo que queda.

Recall@5 y MRR en el 20 % apartado: ¿sale la asignatura aprobada en el top-5?

Precisión estimada: 50 sugerencias revisadas a mano (el histórico no tiene negativos).

Matching de cada asignatura UC3M

- Top-10 asignaturas del destino por coseno.
- Se filtran por el umbral calibrado.
-  **PRECEDENTE_APROBADO** si el par está en el histórico. Se muestra **siempre**, aunque quede por debajo del umbral.
-  **SUGERENCIA_IA** si es un par nuevo por encima del umbral.
- ECTS de ambos lados y aviso si difieren más de un 25 %.
- Opcional: indicio de "mismo libro de referencia" en la bibliografía, solo si mejora Recall@5.

Un JSON como **contrato** entre pipeline e interfaz

El esquema se cierra en la Semana 1, así la UI se construye con datos ficticios desde el principio y no cambia cuando llegan los reales.

```
{
  "grado": "...", "universidad": "...",
  "generado": "2026-10-20",
  "modelo": "multilingual-e5-small",
  "umbral": 0.71,
  "resultados": [{
    "uc3m": {"codigo": "...", "nombre": "...", "ects": 6},
    "destino": {"codigo": "...", "nombre": "...",
      "ects": 6.0, "url": "..."},
    "score": 0.87,
    "etiqueta": "PRECEDENTE_APROBADO",
    "aviso_ects": false
  }]
}
```

Export

`build_static.py` → `public/data/<grado>__<uni>.json` +
`index.json`

Interfaz

Streamlit en Community Cloud: selección de grado y destino, tabla con score, etiqueta y ECTS, y resumen de créditos. No carga el modelo.

PDF

`reportlab` dentro de la app: propuesta formal lista para el tutor.

TIMELINE

8 semanas, tres parejas, una tarea por semana

Datos históricos (Alejandra+Ángela), scraping y matching (Fran+Mercedes), web (Héctor+Víctor). Dentro de la semana nadie depende de otro grupo. Semana 9 de buffer.

Semanas 1-2

Cimientos

Datos (Alejandra+Ángela): EDA + candidatas → piloto; histórico a SQLite

Scraping (Fran+Mercedes): SQLite + scraper UC3M; scraper del destino

Web (Héctor+Víctor): Pydantic + JSON + bocetos + web mínima; web completa desplegada

Piloto elegido · UC3M e histórico en SQLite

Semanas 3-4

Datos del piloto

Datos: reconciliación de códigos; enlace histórico ↔ destino (con Fran+Mercedes)

Scraping: fichas antiguas + destino terminado

Web: PDF; pulir la web y tests

Pares de calibración contados

Semanas 5-6

Matching

Datos: preparar y hacer la calibración; evaluación; 50 sugerencias a mano

Scraping → matching: embeddings + top-K; matcher + export del JSON real

Web: README de usuario, guion de prueba, tests e2e

$\text{Recall@5} \geq 0,80 \cdot \text{precisión} \geq 0,70 \cdot \text{JSON real}$

Semanas 7-8

Producto

Web: integración con el JSON real, PDF real, redespliegue

Datos + Scraping: tests $\geq 70\%$, README, script de refresco

Todos: prueba con 3-5 usuarios y arreglos

Demo publicada · Entrega del MVP

Tareas pequeñas, revisión entre nosotros e IA como **ayuda**

Aprendemos haciendo: cada tarea tiene un entregable, un "hecho cuando" y una pista para empezar. Nadie se queda atascado solo.

Organización

- Reunión semanal de 30 min (viernes): cada pareja enseña su entregable y confirma la siguiente tarea.
- Tres parejas fijas; cada pareja, **una tarea por semana** con su entregable, en GitHub Issues.
- **Nadie espera a otro grupo** dentro de la semana: lo que hace falta de otros está hecho la semana anterior.
- Si una pareja no tiene tarea crítica, hace algo útil que no bloquea a nadie, o descansa.
- Cada tarea en su rama y con Pull Request que revisa otra persona.
- Más de dos días atascado → se cuenta en el grupo.

Herramientas de IA (gratis)

- **Antigravity** (Claude Opus/Sonnet 4.6, Gemini 3.1 Pro): planificar y entender. Cuota semanal: para pensar.
- **OpenCode + Copilot Student**: programar en el día a día.
- **OpenCode + Zen Free**: boilerplate y tests cuando se acaban los créditos.
- **Gemini (AI Plus)**: estudiar los conceptos.

Reglas con la IA

- Pide siempre que te explique lo que ha hecho.
- **No mergees lo que no entiendes.**
- Toda tarea de código lleva tests: son la comprobación.
- Pasos pequeños: una función, pruébala, sigue.
- El repo lleva `AGENTS.md` para que los agentes conozcan el proyecto.

PRÓXIMOS PASOS

Semana 1: tres grupos, tres cimientos

El repo ya está creado. Tres tareas independientes desde el lunes; el viernes, reunión: se presenta el EDA, se elige el piloto y se confirma la Semana 2. Detalle en `docs/plan_trabajo.md`.

Alejandra + Ángela

EDA de los Excel: pares por grado × universidad, problemas del dato; revisar los 6 catálogos candidatos. El viernes se elige el piloto.

Fran + Mercedes

SQLite + scraper UC3M del plan vigente: esquema de la BD, enumeración, descarga con caché, parser con fixtures y tests.

Héctor + Víctor

Pydantic + frontend: modelos de datos, JSON de ejemplo, bocetos de la pantalla y del PDF, web mínima en Streamlit.
Todos: setup personal con PR mergeada.